

Custom Pokémon in pokeemerald-expansion (Developer Guide)

Executive Summary: This report details how to add fully custom Pokémon and mechanics in the **pokeemerald-expansion** (a GBA *Pokémon Emerald* decompilation) project. We cover precise steps, file locations, and code patterns for new species, types, moves, abilities, items, status conditions, battle flags, AI/trainer integration, Pokédex data, TMs/HMs, and asset injection. For each category we give code snippets/templates, tables of relevant files, and workflow tips. We cite the official pokeemerald-expansion documentation and relevant technical sources. We also discuss build/test workflows (using **devkitARM**, **mGBA**, etc.), debugging aids, and CI practices. Diagrams illustrate data flow and entity relationships.

Tools & Environment

Developing for pokeemerald-expansion requires:

- **DevkitPro/DevkitARM:** A GBA toolchain (C compiler, linker) to build the source into a `.gba` file [52†L27-L35] [55†L9-L14] . Follow pokeemerald-expansion's [INSTALL.md] instructions to install devkitARM (via pacman or installer) [52†L27-L35] . Set the `$DEVKITPRO` and `$DEVKITARM` env variables accordingly.
- **Graphics/Sound tools:** Aseprite (or similar) for 2D sprites (16-color indexed PNGs), a palette editor (e.g. APE) for .pal files, **AdvanceMap** for editing maps/encounters, **Porymap/Poryscript** for coordinates/events, and tools like **grit/lz11** for compressing graphics (handled by the build system via `INCGFX_*` macros) [64†L876-L884] .
- **Original ROM:** An official Pokémon Emerald (U) ROM is needed as the base for patching (pokeemerald-expansion assumes compatibility with the Emerald engine).
- **Emulator/Debugger:** mGBA (with its [debugger/GDB support](#)) or NO\$GBA for testing. mGBA's scripting API and Lua support provide advanced debugging/automation [87†L125-L134] .
- **Build System:** The repo uses a Makefile (`make`) to compile all C/ARM code and link data. Use `make -j$(nproc)` for parallel builds; a successful build yields `pokeemerald.gba` [55†L9-L14] .
- **Documentation:** The pokeemerald-expansion [tutorials](#) are our primary source for file paths and code examples [38†L278-L287] [63†L677-L685] .

File Structure Overview

Below is a key-file table for adding Pokémon, moves, etc:

Category	Key File/Location	Content/Purpose (examples)
Species IDs	<code>include/constants/species.h</code>	<code>#define SPECIES_MYMON 1524</code> – add new <code>SPECIES_*</code> at end to avoid renumbering [38†L283-L291] .

Category	Key File/Location	Content/Purpose (examples)
Species Data	<code>src/data/pokemon/species_info.h</code>	<code>gSpeciesInfo[]</code> entries: base stats, types, abilities, etc [38†L321-L329] [65†L962-L970] .
Dex IDs	<code>include/constants/pokedex.h</code>	National/Hoenn Dex enum: add <code>NATIONAL_DEX_MYMON</code> , <code>HOENN_DEX_MYMON</code> [63†L677-L685] .
Dex Sort	<code>src/data/pokemon/pokedex_orders.h</code>	Arrays for alphabetic/weight/height order: insert new Dex index in correct place [63†L764-L772] .
Graphics (gfx)	<code>graphics/pokemon/[mymon]/anim_front.png</code> etc.	2D image assets (front/back sprites, icon, footprint, palettes).
Graphics (include)	<code>src/data/graphics/pokemon.h</code>	<code>INCGFX_*</code> macros including the above assets into the ROM [64†L876-L884] .
Animations	<code>src/data/pokemon_graphics/front_pic_anims.h</code>	(Legacy) AnimCmd definitions for front-sprite frames [64†L912-L922] . Usually auto-included in <code>species_info</code> .
Form Data	<code>src/data/pokemon/form_species_tables.h</code>	Tables listing forms of a species (e.g. Pikachu forms) [70†L1340-L1349] .
Level-up Moves	<code>src/data/pokemon/level_up_learnsets/gen_X.h</code>	<code>static const struct LevelUpMove sMyMonLevelUpLearnset[] = {...}</code> defining moves by level [66†L19-L27] .
TM/HM Moves	<code>src/data/pokemon/teachable_learnsets.h</code>	(Auto-generated) TM/HM/tutor moves list (<code>const u16 sMyMonTeachableLearnset[] = {MOVE_*, ...}</code>) [66†L61-L69] .
Evolutions	<code>src/data/pokemon/evolution.h</code> (macro usage)	<code>EVOLUTION()</code> entries in <code>gSpeciesInfo</code> (.evolutions field) [67†L49-L57] . Specifies evolve-by-level/item.
Species Text	<code>src/data/text/abilities.h</code>	Ability descriptions (for new abilities).
Wild Encounters	<code>src/data/wild_encounters.json</code>	JSON maps of wild Pokémon: add entries with <code>{"species": "SPECIES_MYMON", ...}</code> [70†L1284-L1292] [70†L1309-L1314] .
Trainer Parties	<code>src/data/trainers/*.json</code> or <code>.h</code>	Trainer definitions: include new species in party lists or pools (see docs on Trainer Party Pools [72†L219-L229]).
Items	<code>include/constants/items.h</code> and <code>src/data/items.h</code>	<code>#define ITEM_MYITEM 5001</code> , add entry in items data (effects, price) and text.

Category	Key File/Location	Content/Purpose (examples)
Moves	<code>include/constants/moves.h</code> and <code>src/data/battle_moves.h</code>	<code>#define MOVE_MY_MOVE X</code> , and entry in <code>gBattleMoves[]</code> (power, type, script, etc) 【43†L559-L568】 .
Abilities	<code>include/constants/abilities.h</code> and possibly <code>src/data/abilities.c</code>	<code>#define ABILITY_MYABILITY X</code> , name/description, and battle logic (script/flags).
Statuses	<code>include/constants/battle.h</code>	Define new <code>STATUS_*</code> or <code>STATUS_FIELD_*</code> ; add handling in battle scripts and <code>battle_util.c</code> .

Table: Key files and data for custom content. (E.g. add `SPECIES_MEWTHREE` in `species.h` [【38†L283-L291】](#) ; include its animations/palettes in `pokemon.h` [【64†L876-L884】](#) ; fill in `gSpeciesInfo` fields [【38†L321-L329】](#) [【65†L962-L970】](#) .)

Adding a New Species

1. Define a species ID. In `include/constants/species.h` , add your new constant after all existing ones (e.g. `#define SPECIES_MYMON 1524`). Adjust the `SPECIES_EGG` macro if present. This ensures existing IDs don't shift [【38†L283-L291】](#) .

```
// include/constants/species.h (example diff)
+#define SPECIES_MYMON      1524
+#define SPECIES_EGG      (SPECIES_MYMON + 1)
#define NUM_SPECIES      SPECIES_EGG
```

2. Populate basic data. In `src/data/pokemon/species_info.h` , add an entry in `gSpeciesInfo[]` for your species. Follow the existing struct fields. For example:

```
// src/data/pokemon/species_info.h (snippet)
[SPECIES_MYMON] =
{
    .baseHP      = 80,
    .baseAttack  = 90,
    .baseDefense = 70,
    .baseSpeed   = 60,
    .baseSpAttack = 100,
    .baseSpDefense = 80,
    .types = MON_TYPES(TYPE_FIRE, TYPE_FAIRY),
    .catchRate = 45,
    #if P_UPDATED_EXP_YIELDS >= GEN_8
    .expYield = 200,
    #endif
    .evYield_HP      = 0,
    .evYield_Attack  = 2,
    .evYield_Defense = 0,
```

```

.genderRatio = PERCENT_FEMALE(50),
.eggCycles = 20,
.friendship = 70,
.growthRate = GROWTH_MEDIUM_FAST,
.eggGroups = MON_EGG_GROUPS(EGG_GROUP_FIELD, EGG_GROUP_FAIRY),
.abilities = {ABILITY_STEADFAST, ABILITY_NONE, ABILITY_NONE},
.bodyColor = BODY_COLOR_RED,
.noFlip = FALSE,          // allow horizontal flip of sprite
// Graphics (fill in after adding assets):
.frontPic    = gMonFrontPic_Mymon,
.frontPicSize = MON_COORDS_SIZE(64, 64),
.frontPicYOffset = 0,
.frontAnimFrames = ANIM_FRAMES(
    ANIMCMD_FRAME(0, 1)
),
.frontAnimId = ANIM_V_JOLT,
.frontAnimDelay = 10,
.enemyMonElevation = 0,
.backPic      = gMonBackPic_Mymon,
.backPicSize = MON_COORDS_SIZE(64, 64),
.backPicYOffset = 0,
.backAnimId = BACK_ANIM_CONCAVE_ARC_SMALL,
.palette     = gMonPalette_Mymon,
.shinyPalette = gMonShinyPalette_Mymon,
.iconSprite = gMonIcon_Mymon,
.iconPalIndex = 1,
FOOTPRINT(Mymon),
// Species metadata
.speciesName = _("Mymon"),
.cryId = CRY_MEFIRST, // add corresponding cry (see next step)
.categoryName = _("Mythical"),
.height = 10, // decimeters (1.0 m)
.weight = 250, // hectograms (25.0 kg)
.description = COMPOUND_STRING(
    "A rare creature\n"
    "that embodies\n"
    "the spirit of\n"
    "ancient fire."
),
.pokemonScale = 256,
.pokemonOffset = 0,
.trainerScale = 256,
.trainerOffset = 0,
// Optional flags:
.isMythical = TRUE,
},

```

This snippet (inspired by [38†L321-L329][65†L962-L970]) shows setting **base stats**, **types**, **catch rate**, **abilities**, and so on. The `MON_TYPES`, `MON_EGG_GROUPS`, and other macros come from `include/constants/pokemon.h` and friends. Insert the entry with proper commas and braces in the array.

3. Assign a cry. Convert or select a cry sound and place it (usually in `sound/freesound/cry`), then define `CRY_MYMON` in `include/constants/sndcmd.h` or related. In our species entry we used `CRY_MEFIRST` as placeholder. Consult ["how_to_new_pokemon"] steps or forum guides for sound conversion (GBA `.m4a` format).

4. Pokédex entries and sorting. Add new Dex indices and sort-order:

- In `include/constants/pokedex.h`, insert your species into the National and Hoenn enum lists `【63†L677-L685】` :

```
enum NationalDexOrder {
    ...,
    NATIONAL_DEX_MEWTWO,
    + NATIONAL_DEX_MYMON,
    NATIONAL_DEX_MEW,
    ...,
};
#define NATIONAL_DEX_COUNT NATIONAL_DEX_MYMON
enum HoennDexOrder {
    ...,
    HOENN_DEX_DEOXY,
    + HOENN_DEX_MYMON,
};
#define HOENN_DEX_COUNT (HOENN_DEX_MYMON + 1)
```

- In `src/pokemon.c`, update `sHoennToNationalOrder[]` (if using Hoenn Dex) to map `HOENN→NATIONAL` `【63†L714-L723】` .
- In your `gSpeciesInfo` entry, set `.natDexNum =` `NATIONAL_DEX_MYMON`, `.height`, `.weight`, `.categoryName`, and `.description` as above `【63†L733-L742】` . (Height/weight are in decimeters/hectograms; use `COMPOUND_STRING()` for multi-line text.)
- **Pokédex sorting:** In `src/data/pokemon/pokedex_orders.h`, add your species's NationalDex value in the correct sorted arrays `【63†L764-L772】` . For example, to keep alphabetic order:

```
const u16 gPokedexOrder_Alphabetical[] = {
    ...,
    NATIONAL_DEX_MEW,
    + NATIONAL_DEX_MYMON,
    NATIONAL_DEX_MEWTWO,
    ...,
};
```

Also insert in the weight and height arrays at the proper lexicographic position `【63†L764-L772】` `【63†L774-L782】` `【63†L792-L800】` (comments in that file help with ordering). This ensures Dex sorting by weight/height is correct.

5. Graphics assets. Create a new folder `graphics/pokemon/mymon/` (copy an existing folder for format). Place these files:

- `anim_front.png` – front sprite sheet (typically 2 frames of 64×64 in indexed PNG, ≤15 colors).
- `back.png` – back sprite.
- `normal.pal`, `shiny.pal` – 16-color palette PNGs (14 palette entries + transparency colors).
- `icon.png` – icon (3×3 4bpp tile PNG, uses one of the 6 predefined palettes).
- `footprint.png` – 4×4 1bpp image (black & white).
- (Optional) `overworld.png` and palettes for OW sprite if using the OW system.

Use a graphics editor (indexed mode, 15 color max) to draw these. The tutorial advises merging colors into palettes manually [\[64†L819-L828\]](#) .

Then **register** these graphics in code (so they get compiled into the ROM):

In `src/data/graphics/pokemon.h`, after the last existing entries, add (using the `INCGFX_*` macros) [\[64†L876-L884\]](#) :

```
// src/data/graphics/pokemon.h
+ const u32 gMonFrontPic_Mymon[] = INCGFX_U32("graphics/pokemon/mymon/
anim_front.png", ".4bpp.lz");
+ const u32 gMonBackPic_Mymon[] = INCGFX_U32("graphics/pokemon/mymon/
back.png", ".4bpp.lz");
+ const u16 gMonPalette_Mymon[] = INCGFX_U16("graphics/pokemon/mymon/
normal.pal", ".gbapal");
+ const u16 gMonShinyPalette_Mymon[] = INCGFX_U16("graphics/pokemon/mymon/
shiny.pal", ".gbapal");
+ const u8 gMonIcon_Mymon[] = INCGFX_U8("graphics/pokemon/mymon/
icon.png", ".4bpp");
+ const u8 gMonFootprint_Mymon[] = INCGFX_U8("graphics/pokemon/mymon/
footprint.png", ".1bpp");
```

These macros compress and include the binary graphics into the final ROM. Make sure to wrap any new lines within the appropriate `#if P_FAMILY_...` block (usually no `P_FAMILY` needed since this is a new family).

6. Animations (optional). In modern versions (≥1.11) you can set up animation in `species_info` via `frontAnimFrames/frontAnimId`. For older versions (≤1.10.3) you'd add entries in `src/data/pokemon_graphics/front_pic_anims.h` [\[64†L912-L922\]](#) . For a simple two-frame flip animation, the example adds:

```
// src/data/pokemon_graphics/front_pic_anims.h (v1.10.3-)
static const union AnimCmd sAnim_Mymon_1[] =
{
    ANIMCMD_FRAME(1, 30),
    ANIMCMD_FRAME(0, 20),
    ANIMCMD_END,
};
```

```
...  
SINGLE_ANIMATION(Mymon);
```

Then in `gSpeciesInfo` use `.frontAnimFrames = ANIM_FRAMES(ANIMCMD_FRAME(0,1)); .frontAnimId = ANIM_V_JOLT;` (or a predefined anim) `【65†L962-L970】`. In v1.11+, simply fill `.frontAnimFrames` directly as above.

7. Linking graphics in species_info. Back in your `gSpeciesInfo` entry, reference the graphics added above. As shown in the snippet, set:

```
.frontPic    = gMonFrontPic_Mymon,  
.frontPicSize = MON_COORDS_SIZE(64,64),  
.frontPicYOffset = 0,  
.frontAnimFrames = ANIM_FRAMES(ANIMCMD_FRAME(0,1)),  
.frontAnimId = ANIM_V_JOLT,  
.enemyMonElevation = 0,  
.backPic     = gMonBackPic_Mymon,  
.backPicSize = MON_COORDS_SIZE(64,64),  
.backPicYOffset = 0,  
.backAnimId = BACK_ANIM_CONCAVE_ARC_SMALL,  
.palette     = gMonPalette_Mymon,  
.shinyPalette = gMonShinyPalette_Mymon,  
.iconSprite  = gMonIcon_Mymon,  
.iconPalIndex = 1,  
FOOTPRINT(Mymon),
```

`【65†L962-L970】` `【65†L974-L979】`. Explanation of fields (from docs): `frontPic` / `backPic` point to sprite data, `MON_COORDS_SIZE(w,h)` defines the sprite box, `frontAnimId` / `backAnimId` pick a built-in animation, `palette` / `shinyPalette` the 16-color palettes, `iconSprite/iconPalIndex` the icon, and `FOOTPRINT()` macro sets the footprint (the macro expands to `.footprint = gMonFootprint_Mymon,`) `【65†L974-L979】`.

8. Species flags and extras. Still in `gSpeciesInfo`, you can set optional flags:

```
.abilities = {ABILITY_STEADFAST, ABILITY_NONE, ABILITY_NONE},  
.bodyColor = BODY_COLOR_RED,  
.isMythical = TRUE,           // (e.g.) skip dex if mystical  
.levelUpLearnset = sMymonLevelUpLearnset,  
.teachableLearnset = sMymonTeachableLearnset,  
.evolutions = EVOLUTION({EVO_LEVEL, 36, SPECIES_MYEVOOLVE0}),
```

The docs explain flags like `.isMythical`, `.isRestrictedLegendary`, `.perfectIVCount` `【65†L1070-L1078】`. Add any that apply. After everything, insert a trailing comma and close the struct for your species.

Evolution and Forms

To link evolutions, use the `EVOLUTION(...)` macros in both parent and child entries. For example, if **MyMon** evolves from Mewtwo at level 100:

- In `SPECIES_MEWTWO` entry:

```
.evolutions = EVOLUTION({EVO_LEVEL, 100, SPECIES_MYMON}),
```

【66†L49-L57】 .

- In `SPECIES_MYMON` entry (optional if no further evo): keep `.evolutions = EVOLUTION({EVO_NONE})` .

For item-based or trade evolutions, use `EVO_ITEM` or `EVO_TRADE` in the macro. If your species has alternate forms (like formes or gender differences), update `src/data/pokemon/form_species_tables.h`: add a table listing all form IDs ending with `FORM_SPECIES_END`, and reference it in `gSpeciesInfo` via `.formSpeciesIdTable = sMyMonFormSpeciesIdTable` 【70†L1355-L1364】 .

Form-change mechanics (Mega, Gigamax) use the **form change tables** in the same file. See examples for setting `{FORM_CHANGE_BATTLE_MEGA_EVOLUTION_ITEM, SPECIES_XY_MEGA, ITEM_XYITE}` etc 【70†L1399-L1408】 . New changes must be added to `include/constants/form_change_types.h` .

Learnsets (Moves/TMs)

Level-up moves: Add a static array in `src/data/pokemon/level_up_learnsets/` . For Gen-9 content:

```
// src/data/pokemon/level_up_learnsets/gen_9.h
static const struct LevelUpMove sMymonLevelUpLearnset[] = {
    LEVEL_UP_MOVE( 1, MOVE_TACKLE),
    LEVEL_UP_MOVE( 1, MOVE_EMBER),
    LEVEL_UP_MOVE(10, MOVE_FLAME_WHEEL),
    LEVEL_UP_MOVE(20, MOVE_SHADOW_BALL),
    LEVEL_UP_END
};
```

Then set `.levelUpLearnset = sMymonLevelUpLearnset` in your `species_info` 【66†L19-L27】 【66†L29-L33】 .

If your project's `P_LVL_UP_LEARNSETS` config is lower gen, put the array in the corresponding `gen_X.h` . The build tools will merge these lists (you can ignore the warning at top of the file 【66†L7-L15】).

TMs/HMs/Tutor moves: pokeemerald-expansion typically auto-generates

`sMymonTeachableLearnset[]` via JSON + a Python helper (enabled by default) [66†L61-L69] . To do this, add your move names to `pokemon_species.json` in the `moves` section or edit the JSON in `tools/learnset_helpers/` . On build, the script will produce a `teachable_learnsets.h` with your list. Alternatively, you may disable the helper and manually write:

```
// src/data/pokemon/teachable_learnsets.h
static const u16 sMymonTeachableLearnset[] = {
    MOVE_BRICK_BREAK,
    MOVE_FIRE_BLAST,
    MOVE_PROTECT,
    MOVE_SOLAR_BEAM,
    MOVE_UNAVAILABLE, // list terminator
};
```

Then assign `.teachableLearnset = sMymonTeachableLearnset` [66†L61-L69] [66†L73-L75] .

TM/HM Entries: Ensure any new TM/HM items have definitions in `include/constants/items.h` (e.g. `#define ITEM_TM101 5001`) and are included in item pocket order (later in `src/data/items.h`) so they appear in the Bag. The auto learnset script reads those.

Custom Moves and Abilities

- **Moves:** To add a completely new move, define `#define MOVE_MY_MOVE X` in `include/constants/moves.h` (where X is next free ID). Then add a `gBattleMoves[MOVE_MY_MOVE] = { ... }` entry in `src/data/battle_moves.h` [43†L559-L568] . Fill fields: `name` (`COMPOUND_STRING`), `description`, `effect`, `power`, `accuracy`, `type`, `category`, `target`, `flags` (like `MOVE_FLAG_MAKES_CONTACT`). For new effects, update `include/constants/battle_move_effects.h` and add a script in `data/battle_scripts_1.s` with a new `BattleScript` label (declare it in `include/battle_scripts.h`). Animations go in `data/battle_anim_scripts.s` (or reuse existing).
- **Abilities:** Define in `include/constants/abilities.h`: `#define ABILITY_MYABILITY X` . Add name/description in `src/data/text/abilities.h` . Hook the ability's behavior by editing `battle_*` code (often in `battle_main.c` or new `battle_abilities.c`), and add any flags to `include/constants/ability_constants.h` . (pokeemerald-expansion may require editing only ~5 files for an ability [75†L1-L4] .) For example, a quiet new ability might go in `BattleScript` and linked via an `ABILITY_*` case.

Custom Types/Status/Battle Mechanics

- **Types:** To add a new type, edit `include/constants/types.h` to add `TYPE_MYTYPE` . Extend the type-effectiveness chart in `src/data/type_effectiveness.h` (add rows/columns). Update code in `battle_util.c` if needed. note: expansion already includes Gen9 types (Stellar, etc).
- **Status conditions:** Define new constants in `include/constants/battle.h` (e.g. `STATUS_FROZEN_CORROSION`), provide an enum in `battle.h` and handle it in status scripts. You must add icon/animation for it, and code in `battle_script_commands.c` (for how it ticks down/effects). (No official tutorial; reference GBATEK/GS2D for status codes.)

- **Volatiles & flags:** New battle effects (confusion, flinch, etc) may require adding enum in `include/constants/battle_move_effects.h` and adding a battle script. Custom battle flags (field conditions, ability flags, etc) can be added in `include/constants/battle.h` and respected in `battle_main.c`.

Items

For a new item (Pokéball, TM, etc): Define in `include/constants/items.h` an ID (`#define ITEM_MYITEM 5001` after existing). In `src/data/items.h`, add an `ITEM(MyItem)` struct entry with effect code, description string, price, etc. (Follow existing items as examples.) If it's a TM/HM, mark it in `include/constants/pokemon.h` for tutor/compatibility or in JSON list. Provide an icon in `graphics/items/`, and register via `src/data/graphics/items.h` using `INCGFX_*` (similar to Pokémon sprites, see existing item entries).

AI and Trainers

- **Wild Encounters:** As above, edit `src/data/wild_encounters.json` to place your species on maps `[70†L1284-L1292]` `[70†L1309-L1314]`. The format is JSON: specify `map`, `encounter_rate`, `min_level`, `max_level`, `species`, and (optionally) `form` or `heldItem`.
- **Trainer Parties:** Pokeemerald-expansion supports **Trainer Party Pools** for dynamic team generation `[72†L219-L229]`. To include your Pokémon, either add it to a trainer's static party (in `.json` or `.h`) or define it in a pool. The trainer docs show a `trainers.json` format where you set `Party Size`, `Pool Size`, `Species`, etc `[73†L79-L87]`. For static trainers (like Gym Leaders), edit the corresponding file in `src/data/trainers/`, adding your species to the party. For random battles, you can use the new pool system (`trainers.party / .poolSize`) `[73†L79-L87]` or leave default.
- **AI Behavior:** If your Pokémon has new moves or abilities, consider adding AI flags. The docs (e.g. `ai_flags.md`) explain how to tune AI decisions. For unique mechanics, you may need to extend the AI logic in `src/battle_ai.c` or related files.

Build, Test, and Debug

- **Build:** Run `make` in the project root. The default target builds `pokeemerald.gba` `[55†L9-L14]`. Use parallel builds (`make -jN`) for speed. If building fails due to undefined references, ensure all new IDs (`SPECIES_`, `MOVE_`, `ITEM_`, etc) have matching constants defined `[43†L559-L568]`.
- **Testing:** Pokeemerald-expansion includes debug tools. For example, pressing **Select** on a Pokémon's summary screen opens a **Sprite Debug Menu** to view sprites/animations `[38†L270-L278]`. Use emulators like **mGBA** (supports GDB debugging and Lua scripting) to step through code and set breakpoints. The feature list notes *"Integrated Testing: pinpoint if your custom mechanics have broken something else"* `[58†L1-L4]` – explore built-in test commands (e.g. `TestBattle` scripts) and the "Pokémon Sprite Visualizer" for checking assets.
- **Unit Tests:** The `test/` directory contains C tests for core functions. You can compile these separately (they have their own Makefile) to verify calculations (stats, IVs, catch formula, etc). Integrate running these tests in your CI.
- **Debugging Tips:**

- Use mGBA's debugger: set breakpoints in C (even step through `battle_main.c`), inspect memory (EWRAM/OAM), and call scripts. The mGBA documentation and scripting interface [\[87†L125-L134\]](#) can automate checks (e.g. confirm learnsets, type effectiveness).
- Check the expansion's `config` files to disable features that might conflict (e.g. turn off default new Pokémon with `P_FAMILY_MYMON` flags if needed).
- For missing graphics, missing palette errors appear on build; ensure all `INCGFX` paths are correct and files exist.
- If adding new moves or abilities, verify you added them in all required places (constants, data, scripts). A common mistake is forgetting `#define MOVE_NEW` in `moves.h`, which causes linker errors [\[43†L559-L568\]](#).
- **CI/Automation:** You can script the build with `make` on CI (e.g. GitHub Actions). After building, run the provided tests (`make test` if available) and perhaps launch a headless emulator to ensure it doesn't crash on startup. Use the `TOOLCHAIN` env var to specify devkitARM path [\[52†L121-L124\]](#). Document any patching of the original ROM (if needed) in your repo's instructions.

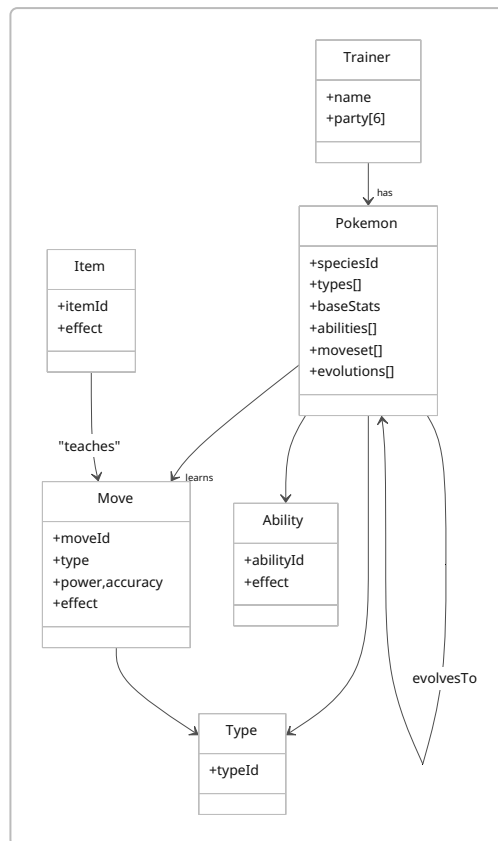
Diagrams

```

flowchart LR
    DevTools[Developer edits code & assets] --> Build[Compile (devkitARM)]
    Build --> ROM[pokeemerald.gba (ROM image)]
    ROM --> Emulator[Run in emulator (mGBA)]
    Emulator --> Tester[Run tests & debug]
    DevTools --> Tools[Graphics/Sound/Map editors (Aseprite, APE, AdvanceMap)]
    DevTools --> Docs[pokeemerald-expansion docs]

```

This flowchart shows the build/test cycle: developer edits sources and assets, compiles into a GBA ROM, and runs it in an emulator for testing. Graphics tools and documentation assist development.



This class diagram highlights relationships: a **Pokémon** has types, stats, abilities, a learnset of **Moves**, and evolution links to other Pokémon. **Moves** have types and effects; **Abilities** define passive effects; **Trainers** hold parties of Pokémon; **Items** can teach moves or trigger evolution.

Compatibility & References

- **pokeemerald-expansion versions:** The above steps assume a recent expansion version (≥ 1.7); some fields or files may differ in older versions. Always check `[[branch]]/docs/tutorials/how_to_new_pokemon.md` for your version. This expansion is *not* bytecode-compatible with official games (it enables gen4–9 features) [\[18†L254-L260\]](#) .
- **Forks:** If using a fork of pokeemerald-expansion (e.g. by other devs), ensure the file paths match. The core concepts remain the same.
- **ROM:** The final output is a hacked GBA ROM; distributing it legally requires only distributing the patch (no Nintendo data). The expansion repo itself only contains disassembled source, not copyrighted ROM.
- **Sources:** We heavily used the official pokeemerald-expansion tutorials and code as references [\[38†L278-L287\]](#) [\[63†L677-L685\]](#) [\[64†L876-L884\]](#) [\[66†L19-L27\]](#) [\[65†L962-L970\]](#) . For low-level GBA details (memory map, CPU), the [\[GBATEK reference\]](#) is invaluable [\[85†L130-L138\]](#) . For emulator debugging, refer to the mGBA website and scripting docs [\[87†L125-L134\]](#) . The **pokecommunity forums** and GitHub issues (e.g. for fix-ups) also contain community insights.

References: Authoritative docs for pokeemerald-expansion and tools are cited throughout: official tutorials and config files [\[38†L278-L287\]](#) [\[63†L677-L685\]](#) [\[64†L876-L884\]](#) [\[43†L559-L568\]](#) , build instructions [\[52†L27-L35\]](#) [\[55†L9-L14\]](#) , and technical references like GBATEK for GBA specs [\[85†L130-L138\]](#) and mGBA for emulation/scripting [\[87†L125-L134\]](#) . These should be consulted for details on each step.
